

From Hops to Vectors: Building AI-Powered Beer Recommendations in Postgres



Pavlo Golub

Senior Developer/Consultant
@ CYBERTEC

Speaker Introduction



Pavlo Golub
Senior Consultant/Developer

- pavlo.golub@cybertec.at
- [@PavloGolub](https://twitter.com/PavloGolub)
- pashagolub.github.io



About CYBERTEC



Why CYBERTEC?



Pure-play
PostgreSQL
Company



Specialised,
International and
Global Team



25 years as a
Key Contributor to
the PostgreSQL
Community



Product &
Service Around
PostgreSQL



Fundamentally
Independent
Operated



AUSTRIA (HQ)

CYBERTEC POSTGRESQL
INTERNATIONAL (HQ)

SWITZERLAND

CYBERTEC POSTGRESQL
SWITZERLAND

URUGUAY

CYBERTEC POSTGRESQL
SOUTH AMERICA

ESTONIA

CYBERTEC POSTGRESQL
NORDIC

POLAND

CYBERTEC POSTGRESQL
POLAND

INDIA

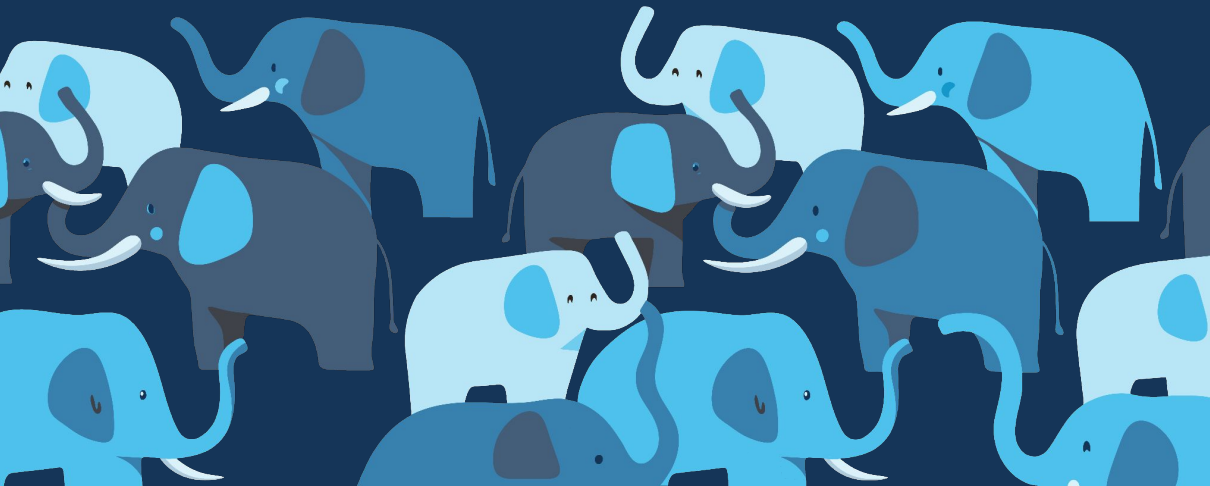
CYBERTEC POSTGRESQL
INDIA PRIVATE LIMITED

SOUTH AFRICA

CYBERTEC POSTGRESQL
SOUTH AFRICA

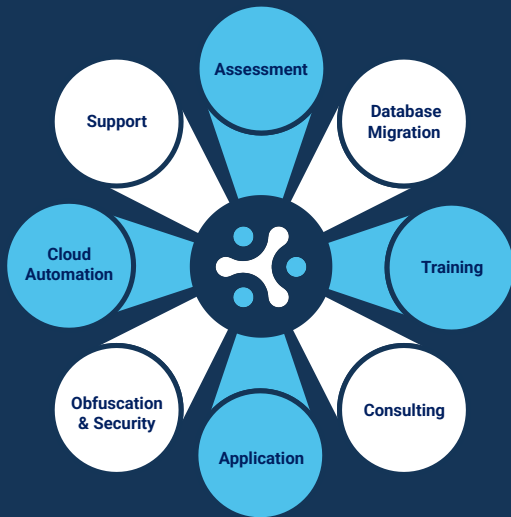


PostgreSQL is at the heart of everything we do, and we are proud to actively contribute to its community and project.

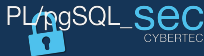


Database Services

- 24/7 Support
- High Availability
- Consulting
- Performance Tuning
- Clustering
- Migration
- Compliance



Database Products & Tools



Agenda

- Why semantic search in PostgreSQL?



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local
- Search like a local: Żywiec, Baltic Porter & pierogi pairings



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local
- Search like a local: Żywiec, Baltic Porter & pierogi pairings
- Under the hood: operators, HNSW, IVFFlat



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local
- Search like a local: Żywiec, Baltic Porter & pierogi pairings
- Under the hood: operators, HNSW, IVFFlat
- The sharp edges: filtering pitfalls & similarity limits



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local
- Search like a local: Żywiec, Baltic Porter & pierogi pairings
- Under the hood: operators, HNSW, IVFFlat
- The sharp edges: filtering pitfalls & similarity limits
- Keeping embeddings fresh with pg_timetable



Agenda

- Why semantic search in PostgreSQL?
- Embeddings in 5 minutes — no math degree required
- **Live demo:** a beer recommendation engine, fully local
- Search like a local: Żywiec, Baltic Porter & pierogi pairings
- Under the hood: operators, HNSW, IVFFlat
- The sharp edges: filtering pitfalls & similarity limits
- Keeping embeddings fresh with pg_timetable
- Q&A — over a (virtual) pint



Why Vectors in Your Database?



The Problem with LIKE '%lemon%'

Classic text search finds **words**, not **meaning**:

```
SELECT beer_name FROM beers  
WHERE info ILIKE '%lemon%';
```



The Problem with LIKE '%lemon%'

Classic text search finds **words**, not **meaning**:

```
SELECT beer_name FROM beers  
WHERE info ILIKE '%lemon%';
```

- Misses “citrusy”, “zesty”, “grapefruit notes”
- No ranking by *how* relevant a match is
- Synonyms, typos, and paraphrases? Forget it



The Problem with LIKE '%lemon%'

Classic text search finds **words**, not **meaning**:

```
SELECT beer_name FROM beers  
WHERE info ILIKE '%lemon%';
```

- Misses “citrusy”, “zesty”, “grapefruit notes”
- No ranking by *how* relevant a match is
- Synonyms, typos, and paraphrases? Forget it

What we actually want

“Show me beers that *taste like* a summer evening by the Vistula” — and get sensible answers.



Embeddings in One Slide

An **embedding model** maps text to a point in high-dimensional space:

```
"crisp citrusy IPA with grapefruit" -> [0.12, -0.48, ..., 0.03]  
                                         \----- 384 numbers -----/
```

- Similar meaning -> nearby points



Embeddings in One Slide

An **embedding model** maps text to a point in high-dimensional space:

```
"crisp citrusy IPA with grapefruit" -> [0.12, -0.48, ..., 0.03]  
                                         \----- 384 numbers -----/
```

- Similar meaning -> nearby points
- Distance between vectors ~ semantic dissimilarity



Embeddings in One Slide

An **embedding model** maps text to a point in high-dimensional space:

```
"crisp citrusy IPA with grapefruit" -> [0.12, -0.48, ..., 0.03]  
                                         \----- 384 numbers -----/
```

- Similar meaning -> nearby points
- Distance between vectors ~ semantic dissimilarity
- Our model: all-MiniLM-L12-v2 — 384 dims, ~130 MB, runs on CPU



Embeddings in One Slide

An **embedding model** maps text to a point in high-dimensional space:

```
"crisp citrusy IPA with grapefruit" -> [0.12, -0.48, ..., 0.03]  
                                         \----- 384 numbers -----/
```

- Similar meaning -> nearby points
- Distance between vectors ~ semantic dissimilarity
- Our model: all-MiniLM-L12-v2 — 384 dims, ~130 MB, runs on CPU
- Free, open (Apache 2.0), no API key, no cloud



pgvector: Vectors as a First-Class Type

```
CREATE EXTENSION vector;  
  
CREATE TABLE beers (  
    id          bigint GENERATED ALWAYS AS IDENTITY,  
    beer_name   text,  
    info        text,  
    embedding   vector(384)  
);
```

- A regular column — joins, WHERE, transactions, backups all work



pgvector: Vectors as a First-Class Type

```
CREATE EXTENSION vector;  
  
CREATE TABLE beers (  
    id          bigint GENERATED ALWAYS AS IDENTITY,  
    beer_name   text,  
    info        text,  
    embedding   vector(384)  
);
```

- A regular column — joins, WHERE, transactions, backups all work
- Distance operators for ORDER BY



pgvector: Vectors as a First-Class Type

```
CREATE EXTENSION vector;  
  
CREATE TABLE beers (  
    id          bigint GENERATED ALWAYS AS IDENTITY,  
    beer_name   text,  
    info        text,  
    embedding   vector(384)  
);
```

- A regular column — joins, WHERE, transactions, backups all work
- Distance operators for ORDER BY
- Two ANN index types: **HNSW** and **IVFFlat**



pgvector: Vectors as a First-Class Type

```
CREATE EXTENSION vector;  
  
CREATE TABLE beers (  
    id          bigint GENERATED ALWAYS AS IDENTITY,  
    beer_name   text,  
    info        text,  
    embedding    vector(384)  
);
```

- A regular column — joins, WHERE, transactions, backups all work
- Distance operators for ORDER BY
- Two ANN index types: **HNSW** and **IVFFlat**
- Your data never leaves your database



The Showcase: Hops'n'Vectors



What We're About to Run

One command. No cloud APIs. No GPU. No Kaggle account.

```
git clone https://github.com/pashagolub/hops-n-vectors.git
cd hops-n-vectors
docker compose up
```

- ~12,600 real beers with descriptions & tasting notes (Kaggle, CC0 — bundled)
- 18 Polish beers included — Żywiec, Tyskie, Okocim, Komes...
- PostgreSQL 18 + pgvector
- Python loader + embedder + TUI
- pg_timetable keeps embeddings in sync

Inspired by *“From Queries to Pints”* (Andrzej Nowicki, FOSDEM 2025)



Architecture

```
+-----+
| docker compose                                     |
| +-----+ +-----+                               |
| | postgres      |<-----| scheduler              | |
| | pgvector :pg18 |      | python + pg_timetable|   |
| | * beers table  |      | 1. load CSV            |   |
| | * vector(384)  |      | 2. embed rows          |   |
| | * HNSW index   |      | 3. run pg_timetable    |   |
| +-----+      | + TUI on demand                |   |
|                  +-----+                       |
+-----+
```

Pipeline is observable in the logs: **validate -> load -> embed -> index -> schedule**



[live] Live: Startup Logs

```
postgres | database system is ready to accept connections
scheduler | validating bundled dataset ... ok
scheduler | loaded 12656 beers (inserted=12656 updated=0)
scheduler | embedding 12656/12656 (100%) in ~8 min
scheduler | CREATE INDEX beers_embedding_hnsw ... done
scheduler | starting pg_timetable ...
```

Call-outs:

- Model downloaded **once**, cached in a Docker volume
- Dataset is bundled — a classic live-demo failure mode, eliminated
- docker compose down (without -v) -> next start in under a minute



Live Demo: Search



[live] The TUI — First Semantic Search

```
docker compose run --rm scheduler tui
```

At the beer> prompt, type:

```
beer> lemon
```

- Citrus, zesty, grapefruit beers rise to the top — no literal “lemon” needed



[live] The TUI — First Semantic Search

```
docker compose run --rm scheduler tui
```

At the beer> prompt, type:

```
beer> lemon
```

- Citrus, zesty, grapefruit beers rise to the top — no literal “lemon” needed
- **Distance** column = cosine distance: 0 is identical, 1 is unrelated



[live] The TUI — First Semantic Search

```
docker compose run --rm scheduler tui
```

At the beer> prompt, type:

```
beer> lemon
```

- Citrus, zesty, grapefruit beers rise to the top — no literal “lemon” needed
- **Distance** column = cosine distance: 0 is identical, 1 is unrelated
- Try :top 10 to widen the net



[live] Search Like You Talk

Free-text prompts, straight to embeddings:

```
beer> crisp refreshing beer for a hot summer day  
beer> dark roasty coffee chocolate  
beer> something to pair with grilled kielbasa and pierogi
```



[live] Search Like You Talk

Free-text prompts, straight to embeddings:

```
beer> crisp refreshing beer for a hot summer day  
beer> dark roasty coffee chocolate  
beer> something to pair with grilled kielbasa and pierogi
```

Why this works

The prompt is embedded with the **same model** as the beer descriptions — query and data live in the same vector space. Many beers carry real pairing data (“Pairs well with: Cured Meats, Pork”) folded into the embedded text — food prompts match it directly.



[live] A Warsaw Special: Polish Beers in the Data

18 of them — yes, they're really in the dataset:

```
SELECT id, beer_name, brewery, style
FROM beers
WHERE info ILIKE '%brewed in poland%'
      OR brewery IN ('Żywiec', 'Okocim', 'Tyskie', 'Lech', 'Komes');
```

Beer	Brewery	Style
Zywiec Lager	Żywiec	Pale Lager
Zywiec Porter	Żywiec	Baltic Porter (since 1881!)
Tyskie Gronie Lager	Tyskie	Pale Lager
Okocim O.K. Beer	Okocim	Pale Lager
Komes Baltic Porter	Komes	Baltic Porter



[live] “More Like This” — `:like <id>`

Find beers similar to Żywiec — k -nearest neighbours of a **row**, not a prompt:

```
beer> :like 10462          -- id of Zywiec Lager from previous query
```

- European pale lagers cluster together — the embedding “knows” the style



[live] “More Like This” — :like <id>

Find beers similar to Żywiec — *k*-nearest neighbours of a **row**, not a prompt:

```
beer> :like 10462          -- id of Zywiec Lager from previous query
```

- European pale lagers cluster together — the embedding “knows” the style
- Same trick in SQL: ORDER BY embedding <=> (SELECT embedding ...)



[live] “More Like This” — :like <id>

Find beers similar to Żywiec — *k*-nearest neighbours of a **row**, not a prompt:

```
beer> :like 10462          -- id of Zywiec Lager from previous query
```

- European pale lagers cluster together — the embedding “knows” the style
- Same trick in SQL: ORDER BY embedding <=> (SELECT embedding ...)
- This is the core of every “customers also liked” feature



[live] Baltic Porter — a Style with Local Roots

Baltic Porter: born around the Baltic Sea, perfected in Poland. **45** of them in the data — including Żywiec Porter (brewed since 1881) and Komes from Browar Fortuna:

```
beer> strong dark baltic porter smooth velvety  
beer> :like 11149          -- Komes Baltic Porter
```



[live] Baltic Porter — a Style with Local Roots

Baltic Porter: born around the Baltic Sea, perfected in Poland. **45** of them in the data — including Żywiec Porter (brewed since 1881) and Komes from Browar Fortuna:

```
beer> strong dark baltic porter smooth velvety  
beer> :like 11149          -- Komes Baltic Porter
```

And a deeper cut — **Grodziskie / Grätzer**, the smoked wheat ale from Grodzisk Wielkopolski:

```
beer> smoked wheat ale historic polish style  
beer> :like 11446          -- Piwo Grodziskie (Prof. Fritz Briem)
```



[live] Peek Behind the Curtain: :explain

```
beer> :explain  
beer> lemon
```

Now every search also prints EXPLAIN ANALYZE:

```
Limit  
  -> Index Scan using beers_embedding_hnsw on beers  
      Order By: (embedding <=> '[...]'::vector)  
Execution Time: ~1 ms
```

The TUI is just SQL. Time to open psql and see how it really works.



Under the Hood



[live] Distance Operators — demo/01_operators.sql

```
SELECT '[1,2,3]':vector <-> '[4,5,6]':vector; -- L2 (Euclidean)
SELECT '[1,2,3]':vector <=> '[4,5,6]':vector; -- cosine <- ours
SELECT '[1,2,3]':vector <#> '[4,5,6]':vector; -- neg. inner product
SELECT '[1,2,3]':vector <+> '[4,5,6]':vector; -- L1 (taxicab)
```

Cosine special cases:

Vectors	Cosine distance
identical	0
orthogonal	1
opposite	2



[live] Baseline: No Index — demo/02_explain_no_index.sql

```
DROP INDEX beers_embedding_hnsw;

EXPLAIN (ANALYZE, BUFFERS)
SELECT beer_name, style
FROM beers
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 10462)
LIMIT 5;                                -- 10462 = Zywiec Lager
```

- Seq Scan on beers — **every** row is read and compared



[live] Baseline: No Index — demo/02_explain_no_index.sql

```
DROP INDEX beers_embedding_hnsw;

EXPLAIN (ANALYZE, BUFFERS)
SELECT beer_name, style
FROM beers
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 10462)
LIMIT 5;                                -- 10462 = Zywiec Lager
```

- Seq Scan on beers — **every** row is read and compared
- Top-N heapsort finds the 5 nearest



[live] Baseline: No Index — demo/02_explain_no_index.sql

```
DROP INDEX beers_embedding_hnsw;

EXPLAIN (ANALYZE, BUFFERS)
SELECT beer_name, style
FROM beers
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 10462)
LIMIT 5;                                -- 10462 = Zywiec Lager
```

- Seq Scan on beers — **every** row is read and compared
- Top-N heapsort finds the 5 nearest
- Fine at 12,600 rows... catastrophic at 12.6 million



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);    -- build-time knobs
```

- Plan flips to Index Scan using beers_embedding_hnsw



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64); -- build-time knobs
```

- Plan flips to Index Scan using beers_embedding_hnsw
- Far fewer buffers, execution time drops dramatically



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);  -- build-time knobs
```

- Plan flips to Index Scan using beers_embedding_hnsw
- Far fewer buffers, execution time drops dramatically
- Same top-5 as brute force — high recall on this dataset



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);  -- build-time knobs
```

- Plan flips to Index Scan using beers_embedding_hnsw
- Far fewer buffers, execution time drops dramatically
- Same top-5 as brute force — high recall on this dataset
- m — links per node in the graph (more = better recall, bigger index)



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64); -- build-time knobs
```

- Plan flips to Index Scan using beers_embedding_hnsw
- Far fewer buffers, execution time drops dramatically
- Same top-5 as brute force — high recall on this dataset
- *m* — links per node in the graph (more = better recall, bigger index)
- *ef_construction* — how hard we search while *building* (higher = slower build, better graph)



[live] HNSW — demo/03_hnsw.sql

Hierarchical Navigable Small World: a graph-based ANN index.

```
CREATE INDEX beers_embedding_hnsw ON beers
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);    -- build-time knobs
```

- Plan flips to Index Scan using `beers_embedding_hnsw`
- Far fewer buffers, execution time drops dramatically
- Same top-5 as brute force — high recall on this dataset
- `m` — links per node in the graph (more = better recall, bigger index)
- `ef_construction` — how hard we search while *building* (higher = slower build, better graph)
- `hnsw.ef_search` — how hard we search while *querying* (the runtime speed vs recall dial)



[live] IVFFlat — demo/04_ivfflat.sql

Inverted-file flat: partition space into clusters, search only some.

```
CREATE INDEX ON beers USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 112);           -- split data into 112 clusters (~ sqrt(12656))

SET ivfflat.probes = 11;       -- search 11 of those clusters per query
```

	HNSW	IVFFlat
Build time	slower	faster
Query speed / recall	better	good
Memory	higher	lower
Needs data before build	no	yes (trains on it)

- `lists` — how many clusters to carve the vectors into (build time)
- `probes` — how many clusters to actually search (query time); more probes = better recall, slower query — demo it live



The Sharp Edges



[live] Filtering Pitfall — demo/05_filtering_pitfalls.sql

ANN indexes fetch candidates **first**, apply WHERE **after**:

```
SELECT beer_name, abv
FROM beers
WHERE abv > 12                -- selective: 190 of 12,656 rows (~1.5%)
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 1)
LIMIT 10;                    -- id 1 = an 8% double IPA
```

- Seq scan: **10 rows** (correct)

The scary part

On demo-sized tables the planner saves you with a seq scan. In production, the ANN index *is* the plan — and this bites for real.



[live] Filtering Pitfall — demo/05_filtering_pitfalls.sql

ANN indexes fetch candidates **first**, apply WHERE **after**:

```
SELECT beer_name, abv
FROM beers
WHERE abv > 12                -- selective: 190 of 12,656 rows (~1.5%)
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 1)
LIMIT 10;                    -- id 1 = an 8% double IPA
```

- Seq scan: **10 rows** (correct)
- HNSW index scan: **far fewer!** — most ef_search candidates had abv <= 12

The scary part

On demo-sized tables the planner saves you with a seq scan. In production, the ANN index *is* the plan — and this bites for real.



[live] Filtering Pitfall — demo/05_filtering_pitfalls.sql

ANN indexes fetch candidates **first**, apply WHERE **after**:

```
SELECT beer_name, abv
FROM beers
WHERE abv > 12                -- selective: 190 of 12,656 rows (~1.5%)
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 1)
LIMIT 10;                    -- id 1 = an 8% double IPA
```

- Seq scan: **10 rows** (correct)
- HNSW index scan: **far fewer!** — most ef_search candidates had abv <= 12
- Silently returns fewer rows than LIMIT — no error, no warning

The scary part

On demo-sized tables the planner saves you with a seq scan. In production, the ANN index *is* the plan — and this bites for real.



[live] Filtering Pitfall — demo/05_filtering_pitfalls.sql

ANN indexes fetch candidates **first**, apply WHERE **after**:

```
SELECT beer_name, abv
FROM beers
WHERE abv > 12                -- selective: 190 of 12,656 rows (~1.5%)
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 1)
LIMIT 10;                    -- id 1 = an 8% double IPA
```

- Seq scan: **10 rows** (correct)
- HNSW index scan: **far fewer!** — most ef_search candidates had abv <= 12
- Silently returns fewer rows than LIMIT — no error, no warning
- Cranking up ef_search helps only partially — and costs latency

The scary part

On demo-sized tables the planner saves you with a seq scan. In production, the ANN index *is* the plan — and this bites for real.



[live] The Fix: Iterative Scan — demo/06_iterative_scan.sql

pgvector >= 0.8: keep scanning until LIMIT is satisfied.

```
SET hnsw.iterative_scan = relaxed_order;

SELECT beer_name, abv
FROM beers
WHERE abv > 12
ORDER BY embedding <=> (SELECT embedding FROM beers WHERE id = 1)
LIMIT 10;                                -- now: all 10 rows OK
```

- relaxed_order — best recall, results may be slightly out of order
- strict_order — exact ordering, slightly slower
- Safety cap: hnsw.max_scan_tuples (default 20,000)



[live] Similarity != Logic — demo/07_similarity_limits.sql

Embeddings capture **topical relatedness**, not polarity:

Pair	ABV	Cosine dist
non-alcoholic IPA vs 120 Minute IPA	0.5% vs 18%	small!
Bud Light vs imperial stout	4.2% vs 18%	not so far
IPA vs IPA (reference)		closest

- Neighbours of a **non-alcoholic** IPA are... regular IPAs



[live] Similarity != Logic — demo/07_similarity_limits.sql

Embeddings capture **topical relatedness**, not polarity:

Pair	ABV	Cosine dist
non-alcoholic IPA vs 120 Minute IPA	0.5% vs 18%	small!
Bud Light vs imperial stout	4.2% vs 18%	not so far
IPA vs IPA (reference)		closest

- Neighbours of a **non-alcoholic** IPA are... regular IPAs
- Why? Both texts talk hops, citrus, tasting notes — same vocabulary



[live] Similarity != Logic — demo/07_similarity_limits.sql

Embeddings capture **topical relatedness**, not polarity:

Pair	ABV	Cosine dist
non-alcoholic IPA vs 120 Minute IPA	0.5% vs 18%	small!
Bud Light vs imperial stout	4.2% vs 18%	not so far
IPA vs IPA (reference)		closest

- Neighbours of a **non-alcoholic** IPA are... regular IPAs
- Why? Both texts talk hops, citrus, tasting notes — same vocabulary
- The model has no concept of negation (“non-”, “alcohol-free”)



[live] Similarity != Logic — demo/07_similarity_limits.sql

Embeddings capture **topical relatedness**, not polarity:

Pair	ABV	Cosine dist
non-alcoholic IPA vs 120 Minute IPA	0.5% vs 18%	small!
Bud Light vs imperial stout	4.2% vs 18%	not so far
IPA vs IPA (reference)		closest

- Neighbours of a **non-alcoholic** IPA are... regular IPAs
- Why? Both texts talk hops, citrus, tasting notes — same vocabulary
- The model has no concept of negation (“non-”, “alcohol-free”)
- **Lesson:** combine vectors with plain SQL filters for hard constraints



Keeping Embeddings Fresh



[live] The pg_timetable Moment

Data changes — embeddings must follow. Automatically.

```
-- 1. Change a description
```

```
UPDATE beers SET info = info || ' extra hoppy' WHERE id = 1;
```

```
-- 2. Trigger the rebuild chain now (or wait for cron */15)
```

```
SELECT timetable.notify_chain_start(chain_id, 'hops-n-vectors-scheduler')  
FROM timetable.chain WHERE chain_name = 'rebuild_embeddings';
```

```
-- 3. Verify: only row 1 was re-embedded
```

```
SELECT id, beer_name, embedded_at  
FROM beers ORDER BY embedded_at DESC LIMIT 5;
```



How Incremental Re-Embedding Works

- Every row stores `text_hash = sha256(composed text)`

Design takeaway

Hash-driven change detection + a database-native scheduler = embeddings that never silently go stale.



How Incremental Re-Embedding Works

- Every row stores `text_hash = sha256(composed text)`
- Rebuild job selects `WHERE text_hash IS DISTINCT FROM sha256(info)`

Design takeaway

Hash-driven change detection + a database-native scheduler = embeddings that never silently go stale.



How Incremental Re-Embedding Works

- Every row stores `text_hash = sha256(composed text)`
- Rebuild job selects `WHERE text_hash IS DISTINCT FROM sha256(info)`
- Only **truly changed** rows are re-embedded — 1 row, not 12,656

Design takeaway

Hash-driven change detection + a database-native scheduler = embeddings that never silently go stale.



How Incremental Re-Embedding Works

- Every row stores `text_hash = sha256(composed text)`
- Rebuild job selects `WHERE text_hash IS DISTINCT FROM sha256(info)`
- Only **truly changed** rows are re-embedded — 1 row, not 12,656
- `pg_timetable PROGRAM` task runs the embedder in the same container

Design takeaway

Hash-driven change detection + a database-native scheduler = embeddings that never silently go stale.



How Incremental Re-Embedding Works

- Every row stores `text_hash = sha256(composed text)`
- Rebuild job selects `WHERE text_hash IS DISTINCT FROM sha256(info)`
- Only **truly changed** rows are re-embedded — 1 row, not 12,656
- `pg_timetable PROGRAM` task runs the embedder in the same container
- No triggers on the hot path, no queue infrastructure, no sidecars

Design takeaway

Hash-driven change detection + a database-native scheduler = embeddings that never silently go stale.



Wrapping Up



Beyond the Demo: Real-World Next Steps

- **Chunking** — split long documents into overlapping passages, embed each

All of it starts with what you saw today: a column, an operator, an index.



Beyond the Demo: Real-World Next Steps

- **Chunking** — split long documents into overlapping passages, embed each
- **Hybrid search** — combine vector similarity with full-text search rank

All of it starts with what you saw today: a column, an operator, an index.



Beyond the Demo: Real-World Next Steps

- **Chunking** — split long documents into overlapping passages, embed each
- **Hybrid search** — combine vector similarity with full-text search rank
- **RAG** — retrieved chunks ground an LLM's answers in *your* data

All of it starts with what you saw today: a column, an operator, an index.



Beyond the Demo: Real-World Next Steps

- **Chunking** — split long documents into overlapping passages, embed each
- **Hybrid search** — combine vector similarity with full-text search rank
- **RAG** — retrieved chunks ground an LLM's answers in *your* data
- **Bigger models** — swap `vector(384)` for 768/1024 dims (schema migration + re-embed)

All of it starts with what you saw today: a column, an operator, an index.



Beyond the Demo: Real-World Next Steps

- **Chunking** — split long documents into overlapping passages, embed each
- **Hybrid search** — combine vector similarity with full-text search rank
- **RAG** — retrieved chunks ground an LLM's answers in *your* data
- **Bigger models** — swap `vector(384)` for 768/1024 dims (schema migration + re-embed)
- **Scale** — partitioning, `halfvec`, binary quantization for millions of rows

All of it starts with what you saw today: a column, an operator, an index.



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY $\Leftrightarrow$



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY $\langle \Rightarrow \rangle$
- HNSW gives you millisecond ANN queries with high recall



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY $\langle \Rightarrow \rangle$
- HNSW gives you millisecond ANN queries with high recall
- Know the sharp edges: post-filtering shortfalls, similarity $\neq$ logic



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY <=>
- HNSW gives you millisecond ANN queries with high recall
- Know the sharp edges: post-filtering shortfalls, similarity != logic
- Iterative scan (pgvector >= 0.8) fixes the filtering pitfall



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY <=>
- HNSW gives you millisecond ANN queries with high recall
- Know the sharp edges: post-filtering shortfalls, similarity != logic
- Iterative scan (pgvector >= 0.8) fixes the filtering pitfall
- pg_timetable keeps embeddings in sync — no external orchestration



Takeaways

- Semantic search is **just SQL** — a vector column and ORDER BY <=>
- HNSW gives you millisecond ANN queries with high recall
- Know the sharp edges: post-filtering shortfalls, similarity != logic
- Iterative scan (pgvector >= 0.8) fixes the filtering pitfall
- pg_timetable keeps embeddings in sync — no external orchestration
- Everything runs locally: your data never leaves PostgreSQL



Try It Yourself

One command to a working demo:

```
git clone \
  github.com/pashagolub/hops-n-vectors
docker compose up
make tui
```

Dataset: CC0 (Public Domain) · Model:

Apache 2.0

Drink responsibly!



Questions?

Dziękuję, Warszawo!

Na zdrowie — and may your queries always use the index.

